SESSION 6 LAB PROGRAMS
COURSE - DESIGN AND ANALYSIS OF ALGORITHMS
CODE - CSA0603

NAME - MARY JEHONNA S
REG.NO: 1965260089

1.

```
#include <stdio.h>
#include <string.h>

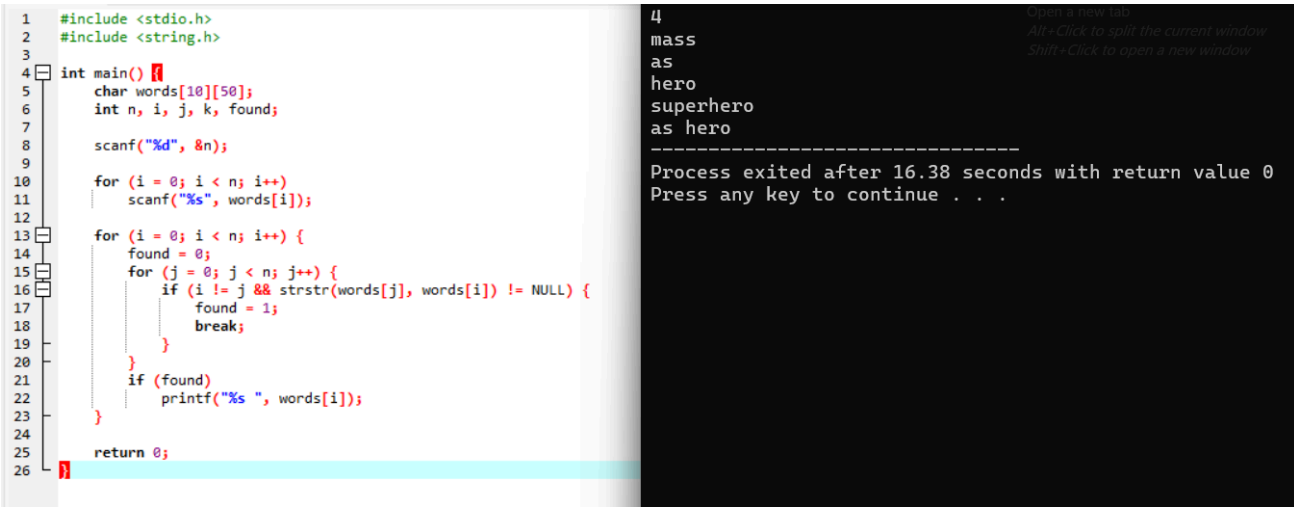
int main() {
   char words[10][50];
   int n, i, j, k, found;

   scanf("%d", &n);

   for (i = 0; i < n; i++)
      scanf("%s", words[i]);

   for (i = 0; i < n; i++) {
      found = 0;
      for (j = 0; j < n; j++) {
         if (i != j && strstr(words[j], words[i]) != NULL) {
            found = 1;
            break;
         }
      }
      if (found)
         printf("%s ", words[i]);
   }

   return 0;
}
```



2.

```
#include <stdio.h>
#include <math.h>

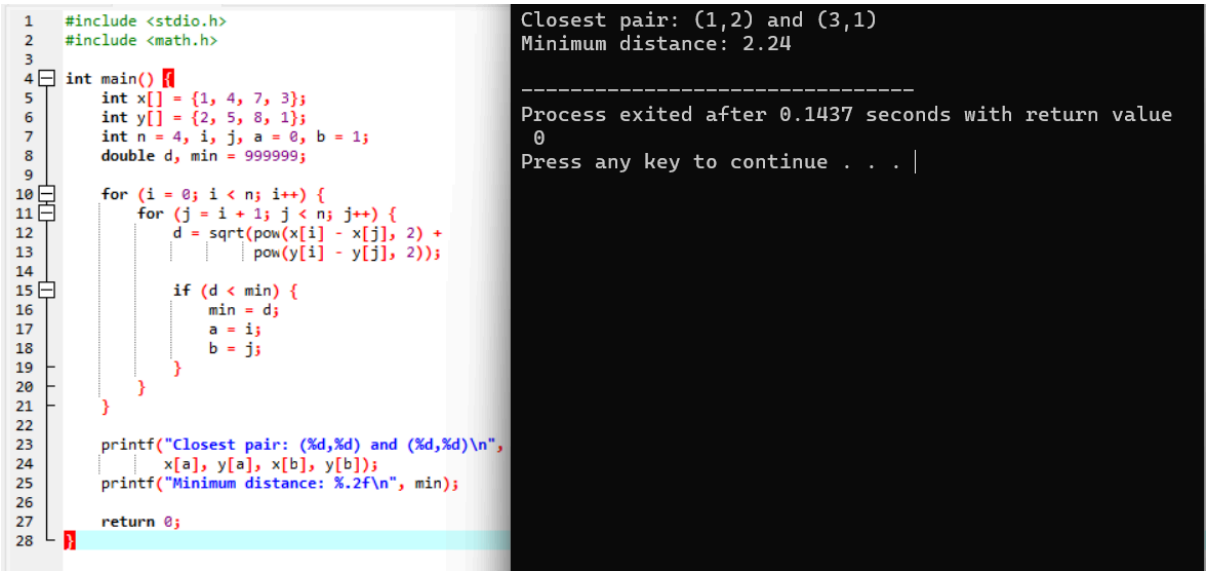
int main() {
   int x[] = {1, 4, 7, 3};
   int y[] = {2, 5, 8, 1};
   int n = 4, i, j, a = 0, b = 1;
   double d, min = 999999;

   for (i = 0; i < n; i++) {
      for (j = i + 1; j < n; j++) {
         d = sqrt(pow(x[i] - x[j], 2) +
               pow(y[i] - y[j], 2));

         if (d < min) {
            min = d;
            a = i;
            b = j;
         }
      }
   }

   printf("Closest pair: (%d,%d) and (%d,%d)\n",
        x[a], y[a], x[b], y[b]);
   printf("Minimum distance: %.2f\n", min);

   return 0;
}
```



3.

```
#include <stdio.h>
#include <math.h>

#define N 8

typedef struct {
   double x, y;
} Point;

int main() {
   Point p[N] = {
      {10,0}, {11,5}, {5,3}, {9,3.5},
      {15,3}, {12.5,7}, {6,6.5}, {7.5,4.5}
   };

   int hull[N] = {0};
   int i, j, k, pos, count = 0;
   double cross, a, b;

   for (i = 0; i < N; i++) {
      for (j = i + 1; j < N; j++) {
         int left = 0, right = 0;

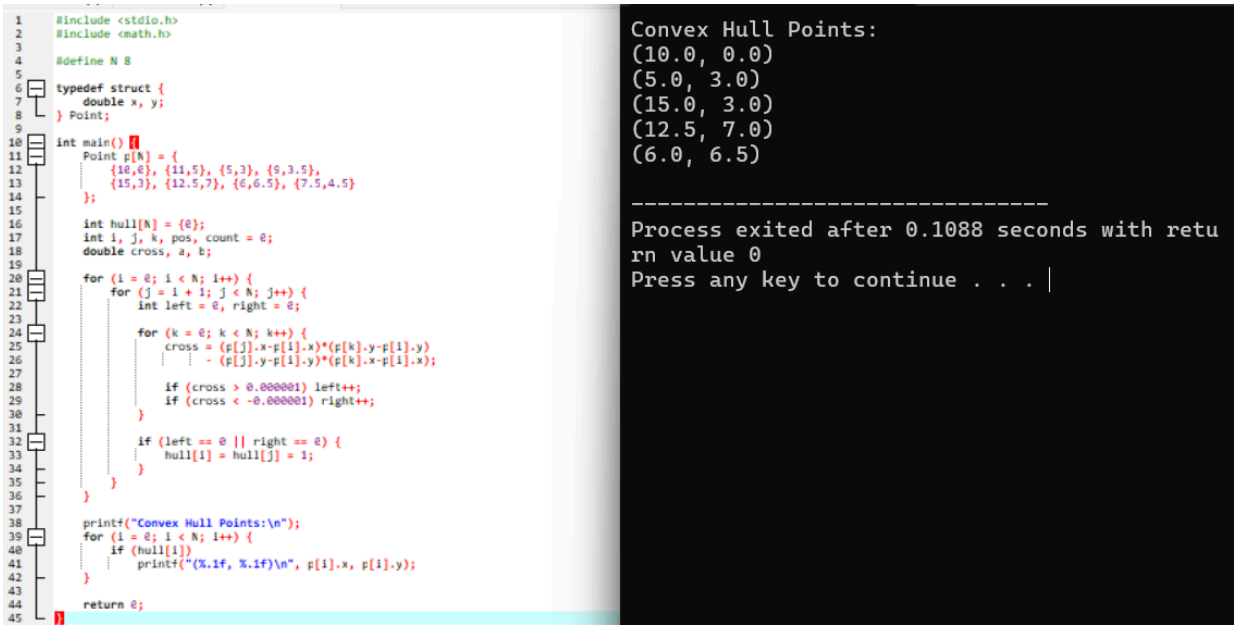
         for (k = 0; k < N; k++) {
            cross = (p[j].x-p[i].x)*(p[k].y-p[i].y)
                 - (p[j].y-p[i].y)*(p[k].x-p[i].x);

            if (cross > 0.000001) left++;
            if (cross < -0.000001) right++;
         }

         if (left == 0 || right == 0) {
            hull[i] = hull[j] = 1;
         }
      }
   }

   printf("Convex Hull Points:\n");
   for (i = 0; i < N; i++) {
      if (hull[i])
         printf("(%.1f, %.1f)\n", p[i].x, p[i].y);
   }

   return 0;
}
```



```
#include <stdio.h>
#include <math.h>

#define N 8

typedef struct {
    double x, y;
} Point;

int main() {
    Point p[N] = {
        {10,0}, {11,5}, {5,3}, {9,3.5},
        {15,3}, {12.5,7}, {6,6.5}, {7.5,4.5}
    };

    int hull[N] = {0};
    int i, j, k, pos, count = 0;
    double cross, a, b;

    for (i = 0; i < N; i++) {
        for (j = i + 1; j < N; j++) {
            int left = 0, right = 0;

            for (k = 0; k < N; k++) {
                cross = (p[j].x-p[i].x)*(p[k].y-p[i].y)
                      - (p[j].y-p[i].y)*(p[k].x-p[i].x);

                if (cross > 0.000001) left++;
                if (cross < -0.000001) right++;
            }

            if (left == 0 || right == 0) {
                hull[i] = hull[j] = 1;
            }
        }
    }

    printf("Convex Hull Points:\n");
    for (i = 0; i < N; i++) {
        if (hull[i])
            printf("(%.1f, %.1f)\n", p[i].x, p[i].y);
    }

    return 0;
}
```

```
Convex Hull Points:
(10.0, 0.0)
(5.0, 3.0)
(15.0, 3.0)
(12.5, 7.0)
(6.0, 6.5)

--------------------------------
Process exited after 0.1088 seconds with retu
rn value 0
Press any key to continue . . .
```

4.

```
#include <stdio.h>

#define N 5

int main() {
    int x[N] = {1, 4, 8, 0, 3};
    int y[N] = {1, 6, 1, 0, 3};
    int i, j, k, left, right;
    double cross;

    printf("Convex Hull Points:\n");

    for (i = 0; i < N; i++) {
        for (j = i + 1; j < N; j++) {
            left = right = 0;

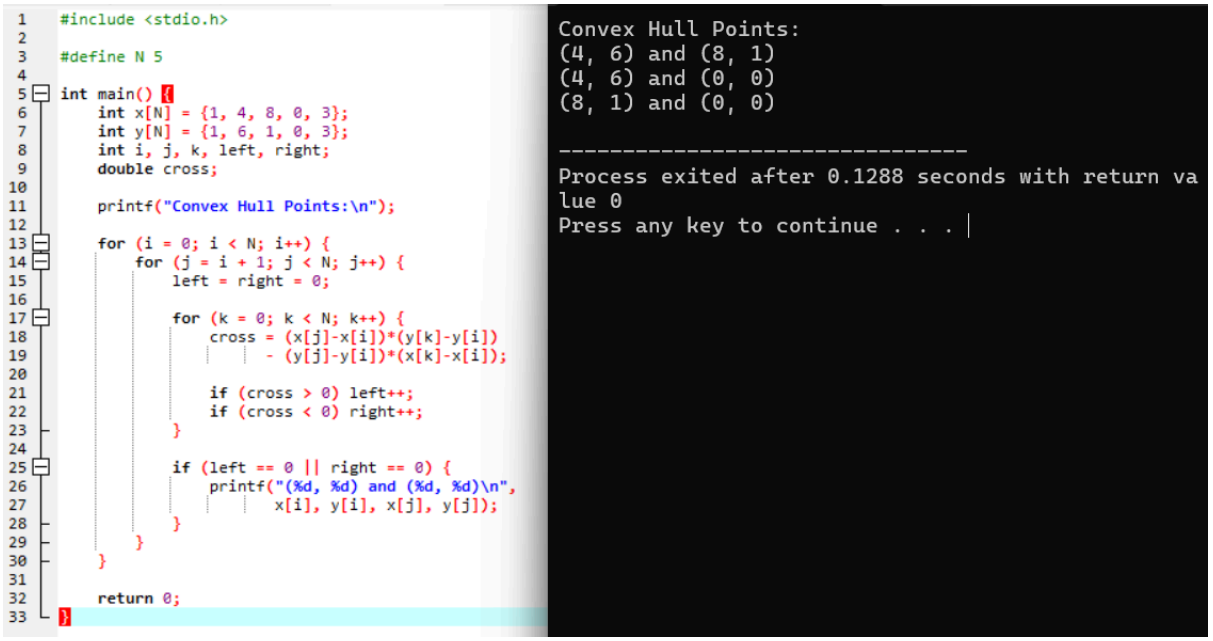
            for (k = 0; k < N; k++) {
                cross = (x[j]-x[i])*(y[k]-y[i])
                      - (y[j]-y[i])*(x[k]-x[i]);

                if (cross > 0) left++;
                if (cross < 0) right++;
            }

            if (left == 0 || right == 0) {
                printf("(%d, %d) and (%d, %d)\n",
                       x[i], y[i], x[j], y[j]);
            }
        }
```

```
    }

    return 0;
}
```



```
#include <stdio.h>

#define N 5

int main() {
    int x[N] = {1, 4, 8, 0, 3};
    int y[N] = {1, 6, 1, 0, 3};
    int i, j, k, left, right;
    double cross;

    printf("Convex Hull Points:\n");

    for (i = 0; i < N; i++) {
        for (j = i + 1; j < N; j++) {
            left = right = 0;

            for (k = 0; k < N; k++) {
                cross = (x[j]-x[i])*(y[k]-y[i])
                      - (y[j]-y[i])*(x[k]-x[i]);

                if (cross > 0) left++;
                if (cross < 0) right++;
            }

            if (left == 0 || right == 0) {
                printf("(%d, %d) and (%d, %d)\n",
                       x[i], y[i], x[j], y[j]);
            }
        }
    }

    return 0;
}
```

```
Convex Hull Points:
(4, 6) and (8, 1)
(4, 6) and (0, 0)
(8, 1) and (0, 0)

--------------------------------
Process exited after 0.1288 seconds with return va
lue 0
Press any key to continue . . .
```

5.

```
#include <stdio.h>
#include <math.h>

#define N 4

int x[N] = {1, 4, 7, 3};
int y[N] = {2, 5, 1, 6};
int path[N], best[N], used[N];
double min = 1e9;

double dist(int a, int b) {
    return sqrt(pow(x[a]-x[b], 2) +
                pow(y[a]-y[b], 2));
}

void tsp(int level, double cost) {
    int i;

    if (level == N) {
        cost += dist(path[N-1], path[0]);

        if (cost < min) {
            min = cost;
            for (i = 0; i < N; i++)
                best[i] = path[i];
```

```
        }
        return;
    }

    for (i = 1; i < N; i++) {
        if (!used[i]) {
            used[i] = 1;
            path[level] = i;
            tsp(level + 1, cost + dist(path[level-1], i));
            used[i] = 0;
        }
    }
}

int main() {
    int i;

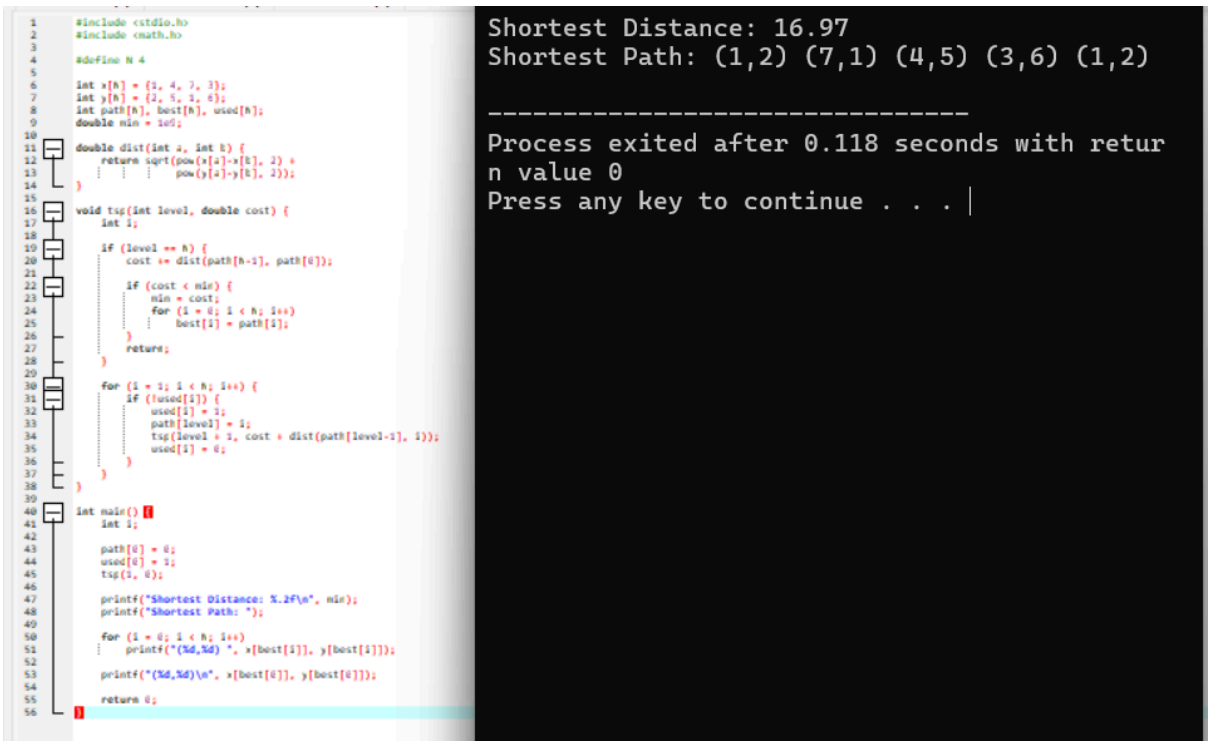
    path[0] = 0;
    used[0] = 1;
    tsp(1, 0);

    printf("Shortest Distance: %.2f\n", min);
    printf("Shortest Path: ");

    for (i = 0; i < N; i++)
        printf("(%d,%d) ", x[best[i]], y[best[i]]);

    printf("(%d,%d)\n", x[best[0]], y[best[0]]);

    return 0;
}
```



6.

```
#include <stdio.h>

int cost[3][3] = {
    {3, 10, 7},
    {8, 5, 12},
    {4, 6, 9}
};

int used[3], assign[3], best[3];
int min = 999999;

void solve(int worker, int total) {
    int j, i;

    if (worker == 3) {
        if (total < min) {
            min = total;
            for (i = 0; i < 3; i++)
                best[i] = assign[i];
        }
        return;
    }

    for (j = 0; j < 3; j++) {
        if (!used[j]) {
            used[j] = 1;
```

```
            assign[worker] = j;
            solve(worker + 1, total + cost[worker][j]);
            used[j] = 0;
        }
    }
}

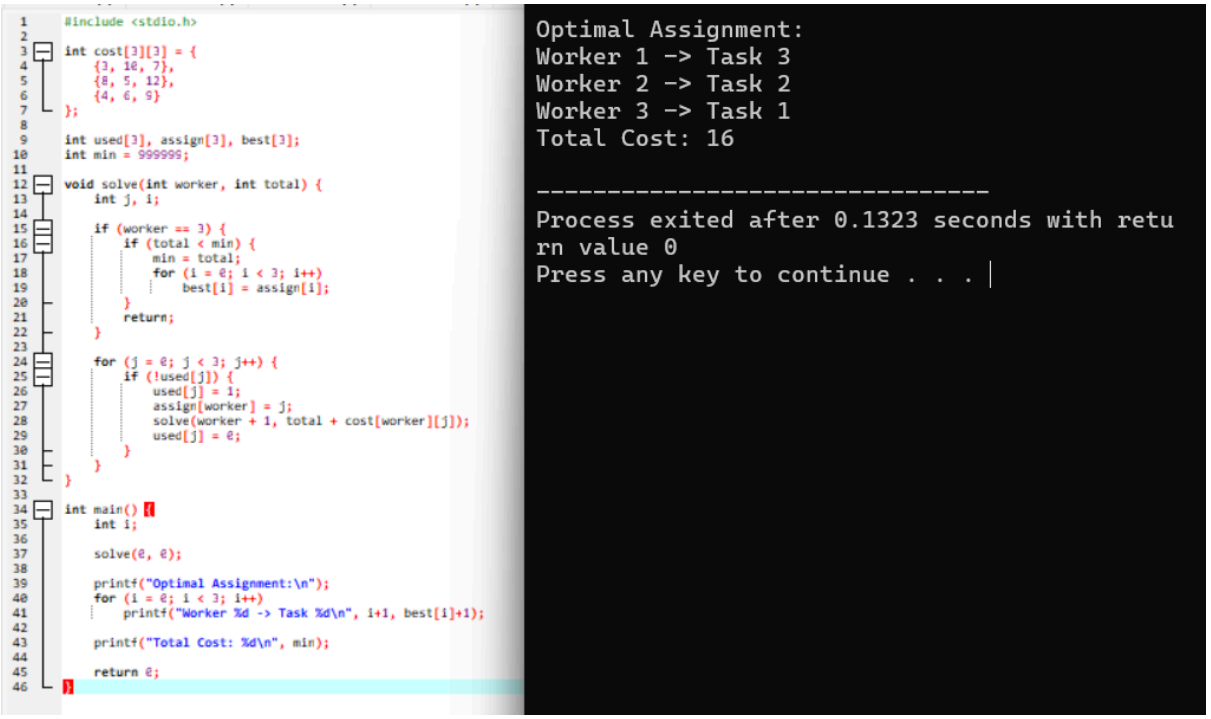
int main() {
    int i;

    solve(0, 0);

    printf("Optimal Assignment:\n");
    for (i = 0; i < 3; i++)
        printf("Worker %d -> Task %d\n", i+1, best[i]+1);

    printf("Total Cost: %d\n", min);

    return 0;
}
```



```
#include <stdio.h>

int cost[3][3] = {
    {3, 10, 7},
    {8, 5, 12},
    {4, 6, 9}
};

int used[3], assign[3], best[3];
int min = 999999;

void solve(int worker, int total) {
    int j, i;

    if (worker == 3) {
        if (total < min) {
            min = total;
            for (i = 0; i < 3; i++)
                best[i] = assign[i];
        }
        return;
    }

    for (j = 0; j < 3; j++) {
        if (!used[j]) {
            used[j] = 1;
            assign[worker] = j;
            solve(worker + 1, total + cost[worker][j]);
            used[j] = 0;
        }
    }
}

int main() {
    int i;

    solve(0, 0);

    printf("Optimal Assignment:\n");
    for (i = 0; i < 3; i++)
        printf("Worker %d -> Task %d\n", i+1, best[i]+1);

    printf("Total Cost: %d\n", min);

    return 0;
}
```

```
Optimal Assignment:
Worker 1 -> Task 3
Worker 2 -> Task 2
Worker 3 -> Task 1
Total Cost: 16

--------------------------------
Process exited after 0.1323 seconds with retu
rn value 0
Press any key to continue . . .
```

7.

```
#include <stdio.h>

int main() {
    int w[] = {2, 3, 1};
    int v[] = {4, 5, 3};
    int n = 3, capacity = 4;
    int i, j, weight, value;
```

```
    int best = 0, mask, bestMask = 0;

    for (mask = 0; mask < (1 << n); mask++) {
        weight = 0;
        value = 0;

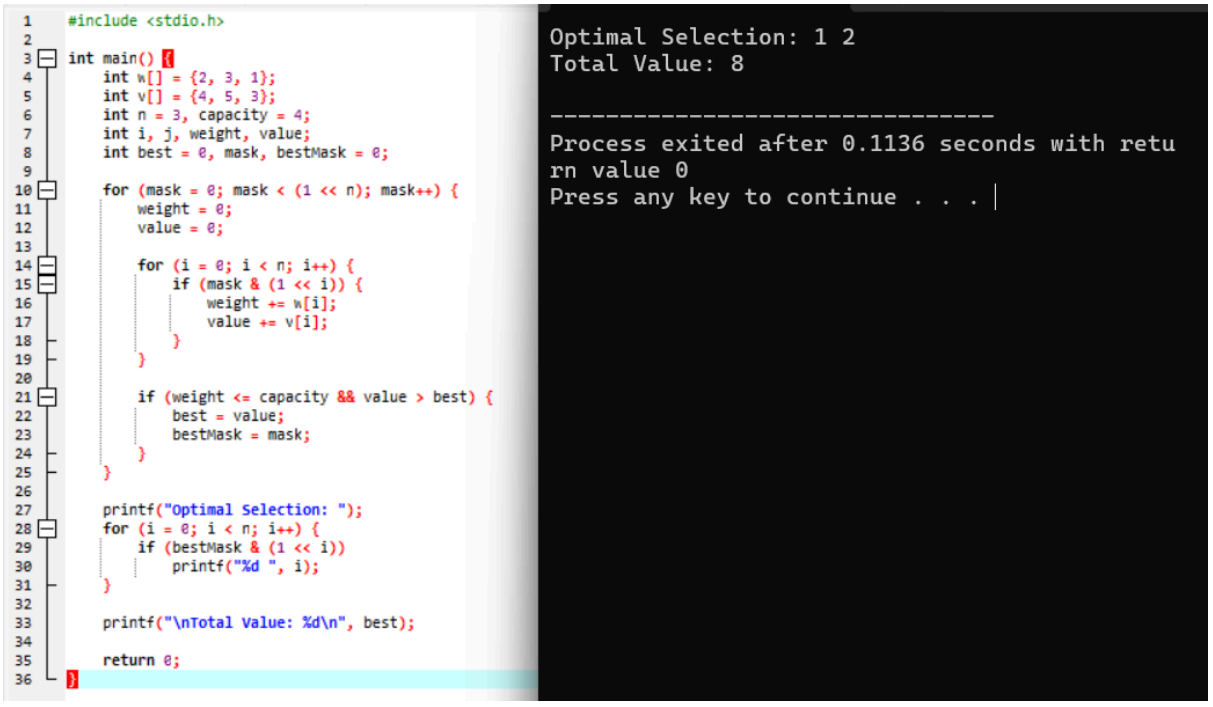
        for (i = 0; i < n; i++) {
            if (mask & (1 << i)) {
                weight += w[i];
                value += v[i];
            }
        }

        if (weight <= capacity && value > best) {
            best = value;
            bestMask = mask;
        }
    }

    printf("Optimal Selection: ");
    for (i = 0; i < n; i++) {
        if (bestMask & (1 << i))
            printf("%d ", i);
    }

    printf("\nTotal Value: %d\n", best);

    return 0;
}
```



```
#include <stdio.h>

int main() {
    int w[] = {2, 3, 1};
    int v[] = {4, 5, 3};
    int n = 3, capacity = 4;
    int i, j, weight, value;
    int best = 0, mask, bestMask = 0;

    for (mask = 0; mask < (1 << n); mask++) {
        weight = 0;
        value = 0;

        for (i = 0; i < n; i++) {
            if (mask & (1 << i)) {
                weight += w[i];
                value += v[i];
            }
        }

        if (weight <= capacity && value > best) {
            best = value;
            bestMask = mask;
        }
    }

    printf("Optimal Selection: ");
    for (i = 0; i < n; i++) {
        if (bestMask & (1 << i))
            printf("%d ", i);
    }

    printf("\nTotal Value: %d\n", best);

    return 0;
}
```

```
Optimal Selection: 1 2
Total Value: 8

--------------------------------
Process exited after 0.1136 seconds with retu
rn value 0
Press any key to continue . . .
```